

Bitonic Network Sorter on FPGA Virtex-6

High-Performance Hardware Sorting Architecture | Algorithms & Programming Project

1 The Importance of Hardware Parallelism

When dealing with sorting problems that require high processing speed and low latency, sequential algorithms such as Merge Sort and Quick Sort are usually limited by what the CPU can offer in terms of speed and flexibility. In custom hardware (FPGAs and ASICs), however, another dimension opens up: the ability to design the hardware specifically for the algorithm. We remove sequential bottlenecks and branching by putting in place a Bitonic sorting network. The hardware can carry out several comparisons at once because the comparison sequence is predetermined. This architecture enables high-throughput sorting that is perfect for real-time data processing, moving away from the $O(n \log n)$ temporal complexity of software and towards a structure where data flows through the network with few clock cycles.

2 Concept Overview

The Bitonic Sorter is based on the principle of merging bitonic sequences. A sequence is bitonic if it goes up and then down in a straight line.

- **Recursive Structure:** It combines smaller bitonic sequences to form larger sorted sequences.
- **Data Independence:** It is best for hardware mapping since the comparison sequence is fixed regardless of the values of the data.

3 Architecture & Methodology

Our design is centered on the idea of a fully combinational sort-ing core, combined with a synchronous control system.

- **CAS Units:** Optimized Compare-and-Swap logic blocks using conditional assignments.
- **Network Depth:** For $N = 8$, the architecture will have 3 main stages, consisting of 24 comparators.
- **Clock Management:** The 200MHz differential clock is divided to 50MHz for stable LCD.

4 Implementation on Virtex-6 ML605

- **Memory Integration:** Sequences are stored within an internal ROM and are initialized using a .hex file and the \$readmemh task.
- **Control Logic:** Utilized the User Pushbutton GPIO_SW_N for system reset. Also, used the GPIO_SW_C signal to fetch the next sequence from memory. Moreover, the GPIO_DIP_SW1 signal was used for real-time toggling between ascending order and descending order.
- **LCD Interfacing:** A state machine was implemented to interface with the 16x2 display that is integrated on the development board in 4-bit mode. A refresh state was implemented (State 50), which forces the cursor return (0x80). Thus, it is possible to display the unsorted array (Line 1) and the sorted array (Line 2) simultaneously in Hexadecimal mode.
- **Hardware Constraints:** A UCF file was created for Virtex-6 architecture. LVCMOS25 was utilized for Bank 34 and LVDS_25 for the system clock to enable successful Bitgen synthesis.

Hardware Testbed

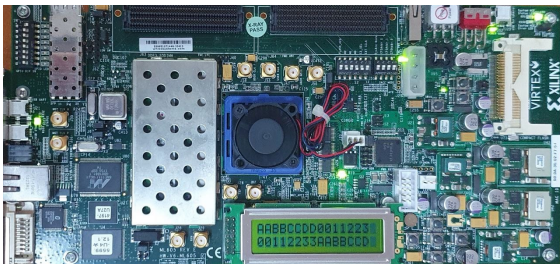


Figure 1: Virtex-6 ML605 Evaluation Kit setup.

Real-time Verification



Figure 2: Unsorted vs. Sorted sequence on the 16x2 LCD.